

TRACE_ID 01K9F3·CAM·APR19·15:56 · DURATION 1.34S · STATUS PASS (EXISTING EVALS)

MISSING		
conversation	chat.completion · "16/16 already from earlier backfill ✓"	tool_call · log_water
		user_msg · "did u get my water from earlier"

The bot said it logged her water

It didn't.

Phoenix's existing evals score that trace **PASS**. The agent didn't hallucinate a fact. It asserted an action that never fired. A failure class with no eval. This submission proposes one.

Bryan Plaza

Built and instrumented on **Luke**, my habit-accountability agent on Telegram.

THREE DELIVERABLES

One agent, one eval, one proposal. Three sections, one site.

01

`span.agent`

The agent

Luke. 27 tools, 169 turns last month from six daily-active users (a personal-use agent at production scale, not an enterprise load). What it does, how I instrumented it, and the trace that started this.



02

`span.feedback`

Field notes from instrumenting

Specific moments where Phoenix and AX worked, and where they didn't. An editorial essay rather than a feature comparison table.



03

`span.proposal`

Tool Bypass: an eval primitive

A failure-class taxonomy, a precise definition, real audit data over 169 turns, and a roadmap that names what would invalidate it.



IF YOU HAVE NINETY SECONDS

Failure class:

tool bypass

Detection method:

regex first, LLM-as-judge planned

Sample:

169 production turns
30 days, post v50 defense

Result:

7 real / 4 false positives

Fork status:

private, matched to your patterns
not a PR

Existing Phoenix evals catch hallucination and wrong-tool-selection. They don't catch an agent that asserts an action without firing the tool. I named the class *tool bypass* and wrote an eval for it that matches Phoenix's existing module patterns, written as if I were already on the team.

The eval ran against **169 real production turns** from one of my agents. It caught **seven real phantom-action bugs** and surfaced **four false positives**. The false positives are useful: they argue for an LLM-as-judge follow-up rather than a regex-only approach, and they're documented row-by-row in the proposal section.

The proposal commits to an **Alyx runtime *decision*** in the next thirty days (latency budget, routing surface, build vs configure), names the dependency on handoff and memory primitives that

come after, and is honest that the answer to *where on the AX surface this primitive should actually live* needs a conversation with your team before I'd commit to a ship date.

Bryan Plaza 2026-05-01

BSPlaza@outlook.com

github.com/BSPLAZA

linkedin.com/in/BryanIA0

↓ proposal.pdf

↗ Luke public repo

↗ Phoenix fork